

Interpolação

Polinômios, estabilidade, Gauss, singulares

Lucas S. Campos

Introdução ao MEC / BEM

“Interpolação”

Gráficos deste capítulo usam **Plots.jl**.

“Objetivos”

1. Montar e resolver o sistema de Vandermonde para interpolação polinomial.
2. Reconhecer quando o polinômio global oscila (Runge) e quando splines por partes são preferíveis.
3. Comparar nós equidistantes e nós de Chebyshev pelo erro máximo em escala log.
4. Aplicar trapézio e Gauss–Legendre e ler a ordem de convergência no gráfico.
5. Suavizar integrandos quase-singulares com transformação (sinh / Monegato).

“Mapa do capítulo”

1. Interpolação polinomial (Vandermonde, exemplo China)
2. Limites do polinômio global e splines por partes
3. Estabilidade, Runge e nós de Chebyshev
4. Integração numérica (trapézio, Gauss)
5. Integrais singulares / quase-singulares
6. Desafio (aleta — ponte para BEM 1D)

“Interpolação polinomial”

Suponha que queremos conhecer a população às vezes entre os anos do censo ou estimar populações futuras. Uma técnica é encontrar um polinômio que passa por todos os pontos de dados.

Dado n pontos $(t_1, y_1), \dots, (t_n, y_n)$, onde os t_i são todos distintos, o problema **de interpolação polinomial** é encontrar um polinômio p de grau menor que n tal que $p(t_i) = y_i$ para todos i .

O problema de interpolação polinomial tem uma solução única. Uma vez encontrado o polinômio interpolador, ele pode ser avaliado em qualquer lugar para estimar ou prever valores.

“Interpolação como um sistema linear”

Dados os dados (t_i, y_i) por $i = 1, \dots, n$, buscamos um polinômio

$$p(t) = c_1 + c_2t + c_3t^2 + \dots + c_nt^{n-1},$$

de tal modo que $y_i = p(t_i)$ para todos i . Essas condições são usadas para determinar os coeficientes $c_1, \dots, c_n$:

$$\begin{aligned} c_1 + c_2t_1 + \dots + c_{n-1}t_1^{n-2} + c_nt_1^{n-1} &= y_1, \\ c_1 + c_2t_2 + \dots + c_{n-1}t_2^{n-2} + c_nt_2^{n-1} &= y_2, \\ c_1 + c_2t_3 + \dots + c_{n-1}t_3^{n-2} + c_nt_3^{n-1} &= y_3, \\ &\vdots \\ c_1 + c_2t_n + \dots + c_{n-1}t_n^{n-2} + c_nt_n^{n-1} &= y_n. \end{aligned}$$

Essas equações formam um sistema linear para os coeficientes c_i :

$$\begin{bmatrix} 1 & t_1 & \dots & t_1^{n-2} & t_1^{n-1} \\ 1 & t_2 & \dots & t_2^{n-2} & t_2^{n-1} \\ 1 & t_3 & \dots & t_3^{n-2} & t_3^{n-1} \\ \vdots & \vdots & & \vdots & \vdots \\ 1 & t_n & \dots & t_n^{n-2} & t_n^{n-1} \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix},$$

ou simplesmente, $\mathbf{Vc} = \mathbf{y}$. A matriz $\mathbf{V}$ é de um tipo especial chamado de Vandermonde.

A interpolação polinomial pode, portanto, ser formulada como um sistema linear de equações com uma matriz de Vandermonde. Criamos dois vetores para dados sobre a população da China. O primeiro tem os anos dos dados do censo e o outro tem a população, em milhões de pessoas.

```
year = [1982,2000,2010,2015];
pop = [1008.18, 1262.64, 1337.82, 1374.62];
t = year .- 1980.0
y = pop;
V = [ t[i]^j for i=1:4, j=0:3 ]
#4x4 Matrix{Float64}:
#1.0  2.0  4.0  8.0
#1.0  20.0  400.0  8000.0
```

```
#1.0 30.0 900.0 27000.0
#1.0 35.0 1225.0 42875.0
c = V \ y
#4-element Vector{Float64}:
# 962.2387878787877
# 24.12775468975476
# -0.592262049062053
# 0.006843867243867301

using Polynomials
p = Polynomial(c)
p(2005-1980)
```

O valor oficial da população para 2005 foi 1303,72, então nosso resultado é bastante bom.

```
using Plots
tt = range(0, 35; length=500)
scatter(t, y; label="real", xlabel="anos desde 1980",
        ylabel="população (milhões)", title="População da China", legend=:topleft)
plot!(tt, p.(tt); label="interpolante", lw=2)
```

“Exercícios”

- Suponha que você queira interpolar os pontos $(-1,0)$, $(0,1)$, $(2,0)$, $(3,1)$ e $(4,2)$ por um polinômio de grau o mais baixo possível. **(a)**  Qual é o grau máximo necessário desse polinômio? **(b)**  Escreva um sistema linear de equações para os coeficientes do polinômio interpolador. **(c)**  Use Julia para resolver numericamente o sistema em (b).
- (a)**  Suponha que você quer encontrar um polinômio cúbico p tal que $p(-1) = -2$, $p'(-1) = 1$, $p(1) = 0$, e $p'(1) = -1$. (Isso é conhecido como um *Interpolador de Hermite*.) Escreva um sistema linear de equações para os coeficientes de p . **(b)**  Use Julia para resolver o sistema linear na parte (a), e faça um gráfico de p sobre $-1 \leq x \leq 1$.

“Continuando com interpolação”

Dado $n + 1$ pontos distintos $(t_0, y_0), (t_1, y_1), \dots, (t_n, y_n)$, com $t_0 < t_1 < \dots < t_n$ chamados de nós, o problema de interpolação é encontrar uma função $p(x)$, chamada de interpolante, tal que $p(t_k) = y_k$ para $k = 0, \dots, n$.

Aqui t_k são os nós e x denota a variável independente contínua.

Nas fórmulas os nós costumam ir de 0 a n . Em Julia os vetores começam em 1: o nó matemático t_k é $t[k+1]$.

Os polinômios são o primeiro candidato óbvio a servir como funções interpolando. Eles são fáceis de trabalhar e, vimos que um sistema linear de equações pode ser usado para determinar os coeficientes de um polinômio que passa por todos os membros de um conjunto de pontos. No entanto, não é difícil encontrar exemplos para os quais a interpolação polinomial leva a resultados inutilizáveis.

Aqui estão alguns pontos que poderíamos considerar ser observações de uma função desconhecida em $[-1, 1]$.

```
using Plots
n = 5
t = range(-1, 1; length=n+1)
y = @. t^2 + t + 0.05*sin(20*t)
scatter(t, y; label="dados", legend=:topleft)
```

O interpolante polinomial, calculado usando o `fit`, parece muito bom.

```
using Plots, Polynomials
p = fit(t, y, n)
xx = range(-1, 1; length=400)
scatter(t, y; label="dados", legend=:topleft)
plot!(xx, p.(xx); label="interpolante", lw=2)
```

Mas agora considere um conjunto diferente de pontos gerados quase exatamente da mesma maneira.

```
n = 18
t = range(-1, 1; length=n+1)
y = @. t^2 + t + 0.05*sin(20*t)
scatter(t, y; label="dados", legend=:topleft)
```

“Continuando com interpolação”

Os pontos em si não têm nada de especial. Mas observe o que acontece com o interpolante polinomial.

```
p = fit(t, y, n)
x = range(-1, 1; length=1000)
scatter(t, y; label="dados", legend=:topleft)
plot!(x, p.(x); label="interpolante", lw=2)
```

Certamente deve haver funções que são mais representativas desses pontos!

Ideia-chave. Polinômio global de grau alto em nós equidistantes pode oscilar violentamente entre os dados (ainda que passe por todos os pontos). Grau baixo **por partes** (spline) costuma representar melhor o mesmo conjunto.

“Interpolação por polinômios por partes”

Para manter pequenos graus de polinomial enquanto interpolam grandes conjuntos de dados, escolheremos interpolantes dos polinômios por partes. Especificamente, o interpolante p deve ser um polinômio em cada subintervalo $[t_{k-1}, t_k]$ para $k = 1, \dots, n$.

Geralmente, designamos antecipadamente um grau máximo para cada parte polinomial de $p(x)$.

```
using Plots, Interpolations

# `t` estritamente crescente (nós). Linear e cúbico bastam para contrastar
# com o polinômio global; o grau 2 não é first-class em Interpolations.jl.
p1 = LinearInterpolation(t, y)           # grau 1 (por partes)
p3 = CubicSplineInterpolation(t, y)     # grau 3 (spline cúbica)
xx = range(first(t), last(t); length=400)
scatter(t, y; label="dados", legend=:topleft)
plot!(xx, p1(xx); label="linear por partes", lw=2)
plot!(xx, p3(xx); label="cúbico por partes", lw=2)
```

“Exercício”

1. Os dois vetores a seguir definem uma forma geometrica.

```
x = [ 0,0.51,0.96,1.06,1.29,1.55,1.73,2.13,2.61,
      2.19,1.76,1.56,1.25,1.04,0.58,0 ]
y = [ 0,0.16,0.16,0.43,0.62,0.48,0.19,0.18,0,
      -0.12,-0.12,-0.29,-0.30,-0.15,-0.16,0 ]
```

Podemos considerar x e y como funções de um parâmetro s , com os pontos sendo valores dados em $s = 0, 1, \dots, 15$ (grade uniforme).

(a) Interpole com spline cúbica (`CubicSplineInterpolation` ou `BSpline(Cubic(...))`) em $x(s)$ e $y(s)$ e plote a curva $(x(s), y(s))$.

(b) O canto à esquerda corresponde a juntar $s = 0$ com $s = 15$. Use contorno **periódico** no parâmetro (grade uniforme):

```
using Interpolations
s = 0:15
itpx = scale(interpolate(x, BSpline(Cubic(Periodic(OnGrid())))), s)
ityp = scale(interpolate(y, BSpline(Cubic(Periodic(OnGrid())))), s)
ss = range(0, 15; length=400)
plot(itpx.(ss), ityp.(ss); label="cúbica periódica", lw=2, aspect_ratio=1)
```

Compare com a spline **sem** Periodic (condição natural).

“Estabilidade da interpolação polinomial”

Escolhemos uma função em relação ao intervalo $[0, 1]$.

```
using Plots, Polynomials
f = x -> sin(exp(2*x))
t = (0:6) ./ 6
y = f.(t)
p = fit(t, y)
xx = range(0, 1; length=400)
plot(xx, f.(xx); label="função", title="Interpolante equidistante, n=6",
      legend=:bottomleft, lw=2)
scatter!(t, y; label="nós")
plot!(xx, p.(xx); label="interpolante", lw=2, ls=:dash)
```

Isso parece bom. Queremos rastrear o comportamento do erro à medida que N aumenta. Estimaremos o erro no interpolante contínuo, amostrando-o em um grande número de pontos e tomando a norma máxima.

```
using LinearAlgebra, Polynomials, Plots
n = 5:5:60; err = zeros(size(n))
x = range(0,1,length=2001)      # pontos para medir o erro
for (i,n) in enumerate(n)
    t = (0:n)/n
    y = f.(t)
    p = fit(t, y)
    err[i] = norm((@. f(x) - p(x)), Inf)
end
using Plots
plot(n, err; yscale=:log10, xlabel="n", ylabel="max error",
      title="Erro de interpolação para nós equidistantes",
      marker=:circle, label=false, lw=2)
```

“Estabilidade da interpolação polinomial”

O erro diminui inicialmente como seria de esperar, mas começa a crescer. Ambas as fases ocorrem a taxas exponenciais em n , ou seja, $O(k^n)$, aparecendo linear em um gráfico semi-log.

“Fenômeno de Runge”

A decepcionante perda de convergência é um sinal de mau condicionamento devido ao uso de nós igualmente espaçados.

```
using Plots, LinearAlgebra
f = x -> 1/(x^2 + 16)
xx = range(-1, 1; length=400)
plot(xx, f.(xx); title="Função teste", label=false, lw=2)
```

Essa função possui infinitamente muitas derivadas contínuas em toda a linha real e parece fácil de aproximar em $[-1, 1]$. Começamos fazendo interpolação polinomial equispacada para alguns pequenos valores de n .

```
using Plots, Polynomials
x = range(-1, 1; length=2501)
plt = plot(xlabel="x", ylabel="|f-p|", yscale=:log10, title="Erro para graus baixos",
           ylims=(1e-20, 1), legend=:bottomright)
for n in 4:4:12
    tt = range(-1, 1; length=n+1)
    pp = fit(tt, f.(tt))
    plot!(plt, x, abs.(f.(x) .- pp.(x)); label="grau $n", lw=2)
end
plt
```

A convergência até agora parece bastante boa, embora não seja uniformemente. No entanto, observe o que acontece à medida que continuamos aumentando o grau.

```
plt = plot(xlabel="x", ylabel="|f-p|", yscale=:log10, title="Erro para graus altos",
           ylims=(1e-20, 1), legend=:bottomright)
for n in @. 12 + 15*(1:3)
    tt = range(-1, 1; length=n+1)
    pp = fit(tt, f.(tt))
    plot!(plt, x, abs.(f.(x) .- pp.(x)); label="grau $n", lw=2)
end
plt
```

“Fenômeno de Runge”

A convergência no meio não pode ficar melhor do que a precisão da máquina em relação aos valores da função. Portanto, a manutenção da lacuna crescente entre o centro e as extremidades empurra as curvas de erro exponencialmente rapidamente nas extremidades, destruindo a convergência.

A observação da instabilidade é o **fenômeno de Runge**: nós igualmente espaçados + grau polinomial crescente $\Rightarrow$ erro explode perto das extremidades do intervalo.

Ideia-chave. A falha não é “polinômios são ruins”, e sim **nós equidistantes em grau alto**. Redistribuir nós (Chebyshev) troca um pouco de precisão no centro por estabilidade global.

Uma família de nós que fornece convergência estável para a interpolação polinomial é os pontos Chebyshev do segundo tipo definido por:

$$t_k = -\cos\left(\frac{k\pi}{n}\right), \quad k = 0, \dots, n.$$

```
x = range(-1, 1; length=2001)
plt = plot(xlabel="x", ylabel="|f-p|", yscale=:log10, title="Erro com os pontos de Chebyshev",
           ylims=(1e-20, 1), legend=:bottomright)
for n in [4, 10, 16, 40]
    tt = [-cos(pi * k / n) for k in 0:n]
    pp = fit(tt, f.(tt))
    plot!(plt, x, abs.(f.(x) .- pp.(x)); label="grau $n", lw=2)
end
plt
```

A partir do grau 16, o erro está dentro da precisão de máquina e ele permanece lá à medida que n aumenta.

“Exercício”

- Para cada caso, calcule o interpolante polinomial usando n nós de Chebyshev do segundo tipo em $[-1, 1]$ por $n = 4, 8, 12, \dots, 60$. Em cada valor de n , calcule o erro (ou seja, $\max |p(x) - f(x)|$ avaliado em 4000 valores de x). Usando uma escala log-linear, plote o erro em função de n e, em seguida, determine uma boa aproximação à constante k de $O(n^{-k})$.

(a) $f(x) = 1/(25x^2 + 1)$ (b) $f(x) = \tanh(5x + 2)$ (c) $f(x) = \cosh(\sin x)$ (d) $f(x) = \sin(\cosh x)$

- Equidistante vs Chebyshev (lado a lado).** Considere

$$f(x) = \frac{1}{1 + 25x^2}$$

em $[-1, 1]$. Para $n = 8, 16, 32$:

- (a) monte o interpolante polinomial com **nós equidistantes**;
- (b) monte o interpolante com **nós de Chebyshev do segundo tipo** $t_k = -\cos(k\pi/n)$.

Em cada n , estime $|f - p|_\infty$ em pelo menos 2000 pontos de teste. Entregue:

- um gráfico de f , p_{eq} e p_{Cheb} para $n = 16$;
- um gráfico (eixo y em escala log) de $|f - p|_\infty$ vs n para as duas famílias de nós;
- uma frase respondendo: **em que regime o nó equidistante ainda é aceitável?**

```
using Plots, Polynomials
f = x -> 1/(1 + 25x^2)
x_test = range(-1, 1; length=2501)
ns = [8, 16, 32]
err_eq = Float64[]
err_ch = Float64[]
for n in ns
    t_eq = range(-1, 1; length=n+1)
    t_ch = [-cos(pi * k / n) for k in 0:n]
    p_eq = fit(collect(t_eq), f.(t_eq))
    p_ch = fit(t_ch, f.(t_ch))
    push!(err_eq, maximum(abs, f.(x_test) .- p_eq.(x_test)))
    push!(err_ch, maximum(abs, f.(x_test) .- p_ch.(x_test)))
end
# exemplo de figura para n = 16
```

```
n = 16
t_eq = range(-1, 1; length=n+1)
t_ch = [-cos(pi * k / n) for k in 0:n]
p_eq = fit(collect(t_eq), f.(t_eq))
p_ch = fit(t_ch, f.(t_ch))
xx = range(-1, 1; length=800)
plot(xx, f.(xx); label="f", lw=2, title="n = 16")
plot!(xx, p_eq.(xx); label="equidistante", ls=:dash, lw=2)
plot!(xx, p_ch.(xx); label="Chebyshev", ls=:dot, lw=2)
plot(ns, err_eq; yscale=:log10, marker=:circle, label="equidistante",
      xlabel="n", ylabel="||f-p|| $\infty$ ", lw=2)
plot!(ns, err_ch; marker=:square, label="Chebyshev", lw=2)
```

“Integração numérica”

A primitiva de e^x é simples, isso torna a avaliação de $\int_{-1}^1 e^x dx$ pelo teorema fundamental trivial.

```
using FastGaussQuadrature, LinearAlgebra
exato = exp(1)-exp(-1)

x, w = gausslegendre(3)
#([-0.7745966692414834, 0.0, 0.7745966692414834], [0.5555555555555556, 0.8888888888888888, 0.5555555555555556])

f(x) = exp(x)

In = dot(w, f.(x))
exato-In
```

A abordagem numérica é muito robusta. Por exemplo, $e^{\sin x}$ não tem primitiva conhecida. Mas numericamente, sua integral não é mais difícil de ser calculada.

```
f(x) = exp(sin(x))
In = dot(w, f.(x))
```

Quando você olha para os gráficos dessas funções, o que é notável é que uma dessas áreas é muito simples, enquanto o outro é analiticamente muito difícil. Do ponto de vista numérico, eles são praticamente o mesmo problema.

```
using Plots
xx = range(-1, 1; length=400)
p1 = plot(xx, exp.(xx); fillrange=0, fillalpha=0.25, label=false,
          xlabel="x", ylabel="exp(x)", ylims=(0, 2.7), lw=2)
p2 = plot(xx, exp.(sin.(xx)); fillrange=0, fillalpha=0.25, label=false,
          xlabel="x", ylabel="exp(sin(x))", ylims=(0, 2.7), lw=2)
plot(p1, p2; layout=(2, 1), size=(700, 500))
```

A integração numérica (**quadratura**) combina valores do integrando amostrados em nós. Nesta seção, primeiro usamos nós igualmente espaçados:

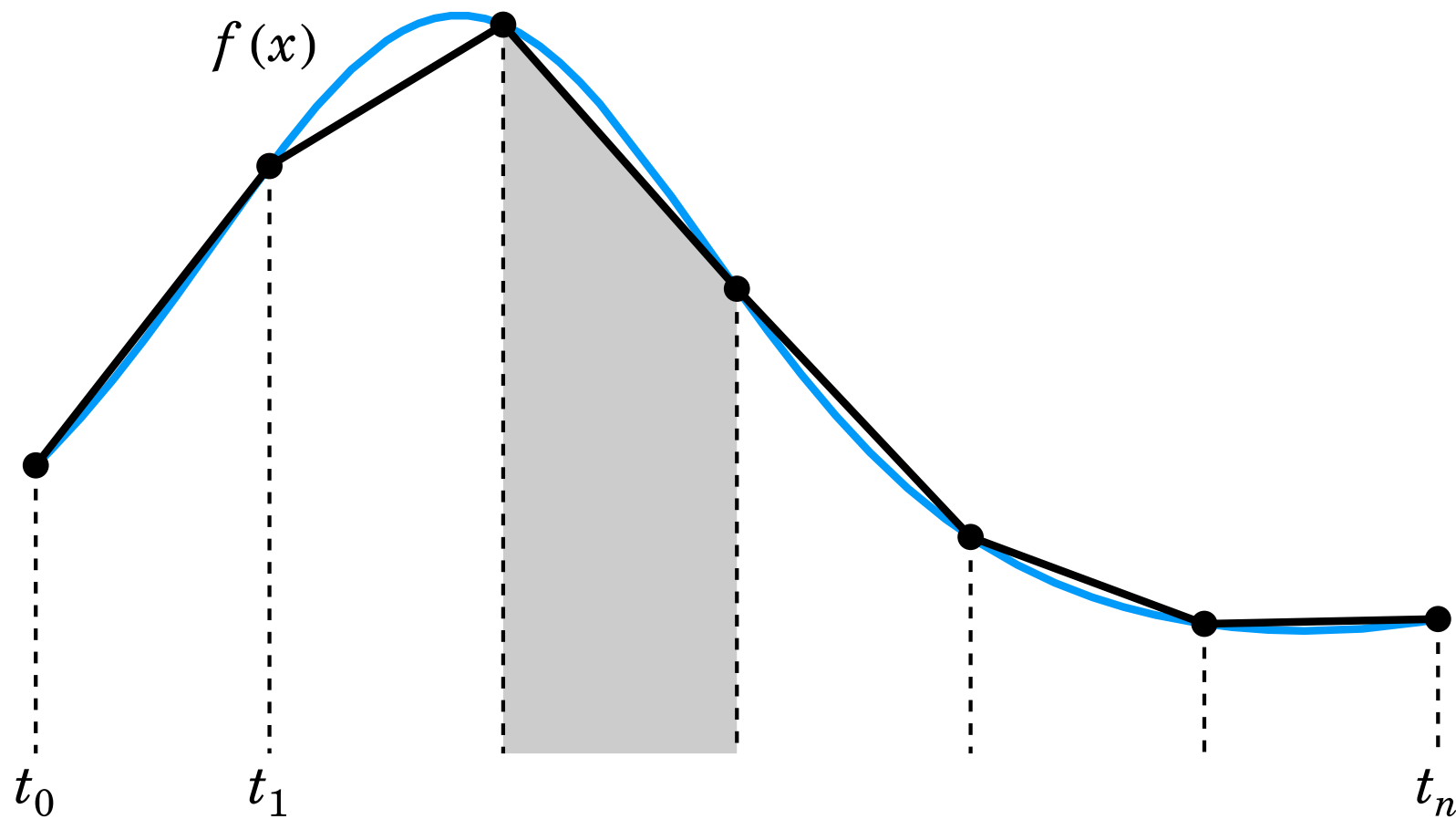
$$t_i = a + ih, \quad h = \frac{b-a}{n}, \quad i = 0, \dots, n.$$

A Integração numérica consiste em uma lista de pesos $w_0, \dots, w_N$ escolhidos de modo que:

$$\int_a^b f(x) dx \approx h \sum_{i=0}^n w_i f(t_i) = h[w_0 f(t_0) + w_1 f(t_1) + \dots w_n f(t_n)].$$

Uma maneira direta de derivar fórmulas de integração é encontrar um interpolante e operar exatamente nele.

Uma das fórmulas de integração mais importantes resulta da integração do interpolante linear por partes. Geometricamente, a fórmula trapezoidal de áreas de trapezoides que se aproximam da região sob a curva $y = f(x)$



Usando áreas de triângulos, é trivial derivar que

“Regra do trapézio”

$$w_i = \begin{cases} 1, & i = 1, \dots, n-1, \\ \frac{1}{2}, & i = 0, n. \end{cases}$$

```
"""
```

```
    trapezoidal(f,a,b,n)
```

Aplique a fórmula de integração do trapezoidal no intervalo [a, b], quebrado em partes iguais. Retorna a estimativa, um vetor de nós e um vetor de valores do integrando nos nós.

```
"""
```

```
function trapezoidal(f,a,b,n)
    h = (b-a)/n
    t = range(a,b,length=n+1)
    y = f.(t)
    T = h * ( sum(y[2:n]) + 0.5*(y[1] + y[n+1]) )
    return T,t,y
end
```

Vamos aproximar $f(x) = e^{\sin 7x}$ no intervalo [0, 2].

```
f = x -> exp(sin(7*x));
a = 0; b = 2;

Integral = 2.6632197827615394 #exato
T,t,y = trapezoidal(f,a,b,40)
@show (T,Integral-T);

n = [ 10^n for n in 1:5 ]
err = []
for n in n
    T,t,y = trapezoidal(f,a,b,n)
```

```
        push!(err,Integral-T)
    end

    foreach(args -> println(args[1], " ", args[2]), zip(n, err))
```

Cada aumento em um fator de 10 em n reduz o erro em um fator de cerca de 100, o que é consistente com a convergência de segunda ordem.

Vamos considerar a fórmula de integração numérica genérica:

$$\int_{-1}^1 f(x)dx \approx \sum_{k=1}^n w_k f(t_k) = Q_n[f],$$

Como existem n nós e n pesos disponíveis para escolher, parece plausível esperar que a quadratura consiga representar exatamente os polinômios de grau $m = 2n - 1$, e essa intuição acaba sendo correta.

Vamos testar isso na integral:

$$\int_{-1}^1 \frac{1}{1+4x^2} dx = \arctan(2).$$

```
f = x->1/(1+4*x^2);
exato = atan(2);

n = 8:4:96
errT = zeros(size(n))
errG = zeros(size(n))
for (k,n) in enumerate(n)
    errT[k] = abs(exato - trapezoidal(f,-1,1,n)[1])
    x, w = gausslegendre(n)
    errG[k] = abs(exato - dot(w, f.(x)))
end

errT[iszero.(errT)] .= NaN
errG[iszero.(errG)] .= NaN
using Plots
plot(collect(n), errT; yscale=:log10, xlabel="nós", ylabel="erro",
     title="integração numérica", ylims=(1e-16, 1),
     marker=:circle, label="trapézio", lw=2)
plot!(collect(n), errG; marker=:circle, label="Gauss-Legendre", lw=2)
```

E agora com uma integral com um integrando mais pontudo:

$$\int_{-1}^1 \frac{1}{1+16x^2} dx = \frac{1}{2} \arctan(4).$$

```
f = x->1/(1+16*x^2);
exato = atan(4)/2;

n = 8:4:96
errT = zeros(size(n))
errG = zeros(size(n))
for (k,n) in enumerate(n)
    errT[k] = abs(exato - trapezoidal(f,-1,1,n)[1])
    x, w = gausslegendre(n)
    errG[k] = abs(exato - dot(w, f.(x)))
end

errT[iszero.(errT)] .= NaN
errG[iszero.(errG)] .= NaN
using Plots
plot(collect(n), errT; yscale=:log10, xlabel="nós", ylabel="erro",
     title="integração numérica", ylims=(1e-16, 1),
     marker=:circle, label="trapézio", lw=2)
plot!(collect(n), errG; marker=:circle, label="Gauss-Legendre", lw=2)
```

“Exercícios”

- Usando a mudança de variável: $z = \phi(x) = a + (b - a)\frac{(x+1)}{2}$ a integral pode ser reescrita como: $\int_a^b f(z) dz = \frac{b-a}{2} \int_{-1}^1 f(\phi(x)) dx$. Sabendo disso, use a quadratura de Gauss para integrar $\int_{\pi/2}^{\pi} x^2 \sin 8x dx = -\frac{3\pi^2}{32}$. Mostre resultados para diferentes valores de n até que se obtenha uma convergência de 10 dígitos.
- Integre numericamente a função $f(x) = -x \log(|x - 1/2|)$ no intervalo $[0, 1/2]$ usando a quadratura de Gauss. Use 5, 10, 15 e 20 pontos de Gauss.

$$I_a = \int_0^{1/2} -x \log\left(\left|x - \frac{1}{2}\right|\right) dx = \frac{1}{16}(3 + 2 \log 2) = 0,274143$$

“Integrais singulares ou quasi-singulares”

A última função integrada tem uma singularidade quando $x = 1/2$. Mesmo com essa singularidade, e tendendo a infinito nesse ponto, essa função pode ser integrada analiticamente. Numericamente é interessante usar técnicas especiais para tratar de integrais desses tipo.

Considere uma integral do tipo:

$$I_s = \int_{-1}^1 \frac{g(\xi)}{(\xi - a)^2 + b^2} d\xi$$

Uma ideia para avaliar numericamente integrais singulares($b = 0$) ou quasi-singulares($b \neq 0$) é encontrar uma transformação cujo Jacobiano seja zero ou bem próximo de zero no ponto singular. Por exemplo, para integrais quasi-singulares podemos usar:

$$\xi = a + b \sinh(\mu s - \eta)$$

onde

$$\begin{aligned} \mu &= \frac{1}{2} \left\{ \operatorname{arcsinh} \left(\frac{1+a}{b} \right) + \operatorname{arcsinh} \left(\frac{1-a}{b} \right) \right\} \\ \eta &= \frac{1}{2} \left\{ \operatorname{arcsinh} \left(\frac{1+a}{b} \right) - \operatorname{arcsinh} \left(\frac{1-a}{b} \right) \right\} \end{aligned}$$

e seu jacobiano é dado por:

$$\frac{d\xi}{ds} = b\mu \cosh(\mu s - \eta)$$

Aplicando essa transformação, obtém-se:

$$I_s = \int_{-1}^1 \frac{g(s(\xi))}{(s(\xi) - a)^2 + b^2} \frac{d\xi}{ds} ds$$

Essa transformação anula (ou reduz) o Jacobiano perto da singularidade, suaviza o integrando e acelera a convergência da quadratura — o mesmo espírito das integrais singulares do BEM.

Ponte com o BEM. Soluções fundamentais geram núcleos $\log r$, $1/r$, etc. no contorno. Transformações do tipo $\sinh$ / Monegato–Sloan (interior) / Sato (extremo) são o dia a dia da montagem de H e G .

```
function sinhtrans(u, a, b)
    μ = 1 / 2 * (asinh((1 + a) / b) + asinh((1 - a) / b))
    η = 1 / 2 * (asinh((1 + a) / b) - asinh((1 - a) / b))

    x = a .+ b * sinh.(μ * u .- η)
    J = b * μ * cosh.(μ * u .- η)
    x, J
end
```

using Plots

"""

plot_quasising_sinh(a=0.0, b=0.05; g=nothing, n=800)

Mostra o integrando quase-singular em $\xi \in [-1,1]$

$f(\xi) = g(\xi) / ((\xi - a)^2 + b^2)$

e o integrando *transformado* na variável $s \in [-1,1]$

$f(\xi(s)) \cdot |d\xi/ds|$

com a transformação `sinhtrans`. O pico alto e estreito vira uma curva bem mais regular (o Jacobiano anula-se perto da quase-singularidade).

"""

```
function plot_quasising_sinh(a=0.0, b=0.05; g=nothing, n=800)
```

```
    g === nothing && (g = ξ -> 1.0)
```

```
    f(ξ) = g(ξ) / ((ξ - a)^2 + b^2)
```

```
    ξ = range(-1, 1; length=n)
```

“Integrais singulares ou quasi-singulares”

```

s = range(-1, 1; length=n)
ξs, J = sinhtrans(collect(s), a, b)
before = f.(ξ)
after  = @. f(ξs) * abs(J)

p1 = plot(ξ, before; lw=2, label="f(ξ)",
          xlabel="ξ", ylabel="integrando",
          title="Antes (quase-singular)", legend=:topright)
vline!(p1, [a]; ls=:dash, color=:gray, label="a = $a")

p2 = plot(s, after; lw=2, label="f(ξ(s)) · |J|",
          xlabel="s", ylabel="integrando transformado",
          title="Depois (× Jacobiano sinh)", legend=:topright)
# marca s* tal que ξ(s*) ≈ a (J mínimo)
_, i = findmin(abs.(ξs .- a))
vline!(p2, [s[i]]; ls=:dash, color=:gray, label="ξ(s) ≈ a")

plot(p1, p2; layout=(1, 2), size=(900, 350))
end

# mapa ξ(s) e Jacobiano
x = range(-1, 1; length=100)
xt, jac = sinhtrans(x, 0.5, 0.01)
p1 = plot(collect(x), xt; xlabel="s", ylabel="ξ", label=false, lw=2)
p2 = plot(collect(x), jac; xlabel="s", ylabel="dξ/ds", label=false, lw=2)
plot(p1, p2; layout=(2, 1), size=(700, 500))

# integrando antes × depois
plot_quasising_sinh(0.0, 0.05)
plot_quasising_sinh(0.5, 0.02) # pico deslocado

```

aplicando essa técnica para a integral do último exemplo pode-se observar uma redução significativa do erro para a mesma quantidade de pontos.

“Integrais singulares ou quasi-singulares”

```
f = x->1/(1+16*x^2);
exato = atan(4)/2;

n = 8

x, w = gausslegendre(n)
xt,J= sinhtrans(x, 0, 1/4)
errG = abs(exato - dot(w, f.(x)))
errGt = abs(exato - dot(w, J.*f.(xt)))
```

1. **Quase-singular + sinhtrans (tabela $b \downarrow$).** Considere

$$I(b) = \int_{-1}^1 \frac{1}{\xi^2 + b^2} d\xi, \quad a = 0,$$

com valor exato

$$I(b) = \left(\frac{1}{b}\right) \left[\arctan\left(\frac{1}{b}\right) + \arctan\left(\frac{1}{b}\right) \right] = \left(\frac{2}{b}\right) \arctan\left(\frac{1}{b}\right).$$

Para $b \in \{10^{-1}, 10^{-2}, 10^{-3}\}$ e Gauss–Legendre com $n \in \{8, 16, 32\}$:

1. calcule o erro absoluto **sem** transformação (integrando amostrado direto em ξ);
2. calcule o erro **com** `sinhtrans(s, 0, b)`, usando o integrando $f(\xi(s)) |J(s)|$;
3. monte uma tabela $(b, n, e_{\text{cru}}, e_{\text{sinh}})$;
4. para o menor b , gere `plot_quasising_sinh(0, b)` e comente o achatamento do pico.

Ponte BEM: b pequeno imita fonte **perto** do elemento (integral quase-singular na montagem de H e G).

```
using FastGaussQuadrature, LinearAlgebra, Plots
Iex(b) = (2/b) * atan(1/b)
f(ξ, b) = 1/(ξ^2 + b^2)

bs = (1e-1, 1e-2, 1e-3)
```

```
ns = (8, 16, 32)
println("b\tn\terr_cru\terr_sinh")
for b in bs, n in ns
    s, w = gausslegendre(n)
    # sem transformação
    e_cru = abs(Iex(b) - dot(w, f.(s, b)))
    # com sinhtrans (a = 0)
    ξ, J = sinhtrans(s, 0.0, b)
    e_sinh = abs(Iex(b) - dot(w, f.(ξ, b) .* abs.(J)))
    println("$b\t$n\t$e_cru\t$e_sinh")
end
plot_quasising_sinh(0.0, 1e-3)
```

“Exercício”

1. Integre de novo $f(x) = -x \log(|x - 1/2|)$ em $[0, 1/2]$ com Gauss, agora usando a transformação abaixo. A singularidade está no **extremo** $x = 1/2$: mapeie o intervalo para $[-1, 1]$ (afim) e chame Monegato com $s_0 = 1$ (ramo Sato). Compare graus $m = 3, 4, 5$ e $n = 5, 10, 15, 20$ pontos de Gauss com o valor I_a do exercício anterior. (Opcional: repita com uma singularidade **interior** artificial, p.ex. estendendo o intervalo, e graus **ímpares** $q = 3, 5$.)

```

"""
    Monegato(t, s0, q=5)

Transformação em `t ∈ [-1,1]` → `s ∈ [-1,1]`, com Jacobiano `ds/dt`.

- `|s0| < 1` (interior): Monegato–Sloan de grau *ímpar* `q ≥ 3`.
- `s0 ≈ ±1` (extremo): Sato de grau `q ≥ 2` (par ou ímpar).

Retorna `(s, ds)`.
"""
function Monegato(t, s0, q::Integer=5; atol=1e-14)
    t = float(t)
    s0 = float(s0)
    # --- extremo: Sato ---
    if isapprox(s0, 1; atol=atol)
        q ≥ 2 || throw(ArgumentError("Sato no extremo +1 exige q ≥ 2"))
        # Φ(t) = 1 - (1-t)^q / 2^(q-1)    (suaviza s = +1)
        u = @. 1 - t
        s = @. 1 - u^q / 2^(q - 1)
        ds = @. q * u^(q - 1) / 2^(q - 1)
        return s, ds
    elseif isapprox(s0, -1; atol=atol)
        q ≥ 2 || throw(ArgumentError("Sato no extremo -1 exige q ≥ 2"))
        # espelho: suaviza s = -1
        u = @. 1 + t
        s = @. -1 + u^q / 2^(q - 1)
        ds = @. q * u^(q - 1) / 2^(q - 1)
        return s, ds
    end
end

```

```

end
# --- interior: Monegato–Sloan ---
(-1 < s0 < 1) || throw(ArgumentError("s0 deve estar em (-1,1) ou ±1"))
(q ≥ 3 && isodd(q)) || throw(ArgumentError(
    "Monegato–Sloan interior exige grau ímpar q = 3,5,7,... (recebeu q=$q)"))
aq = (1 + s0)^(1 / q)
bq = (1 - s0)^(1 / q)
δ = (1 / 2)^q * (aq + bq)^q
t0 = (aq - bq) / (aq + bq)
u = @. t - t0
s = @. s0 + δ * u^q
ds = @. q * δ * u^(q - 1)
return s, ds
end

```

1. Use 10 pontos de Gauss para calcular as integrais abaixo. Compare com a solução analítica. Quais integrais tiveram os maiores erros? Por que?

144.04761904761904

5.090354888959125 $\mathrm{d}x$, $-\frac{2}{3} = -0.666666666666667$; $\int_0^\pi (\sin(3x) \cdot \cos(2x) + 2x^3 + 3x \sin(x)) \mathrm{d}x$, 59.3293234777059; $\int_0^1 \frac{1}{x+1} \mathrm{d}x$, $\ln(2) = 0.6931471805599453$; $\int_0^1 \frac{1}{x+0.1} \mathrm{d}x$, $\ln(11) = 2.397895272798371$

0.2958368660043291

“Desafio”

Considere o resfriamento de uma aleta circular por meio de transferência de calor por convecção ao longo de seu comprimento. A convecção dá origem a uma perda de calor ou termo de sumidouro dependente da temperatura na equação governante. Mostrada na Figura está uma aleta cilíndrica com área de seção transversal uniforme A . A base está a uma temperatura de 100°C (T_B) e a extremidade direita está isolada. A aleta está exposta a uma temperatura ambiente de 20°C . A transferência de calor unidimensional nesta situação é governada por

$$\frac{d}{dx}\left(kA\frac{dT}{dx}\right) - hP(T - T_{\infty}) = 0$$

onde h é o coeficiente de transferência de calor por convecção, P o perímetro, k a condutividade térmica do material e T_{∞} a temperatura ambiente. Calcule usando BEM a distribuição de temperatura ao longo da aleta e compare os resultados com a solução analítica fornecida por

$$\frac{T - T_{\infty}}{T_B - T_{\infty}} = \frac{\cosh[n(L - x)]}{\cosh(nL)}$$

